

Funciones de Activación

Asignatura: Tecnologías Emergentes

Autor(a): Barzola Chancasanampa Marlit

22 de septiembre de 2026

1. Introducción

Las funciones de activación introducen la no-linealidad necesaria para que una red neuronal pueda aprender patrones complejos y solucionar problemas no separables linealmente. A continuación, se describen tres funciones de activación: **Tanh**, **LeakyReLU** y **GELU**.

1.1. 1. Tangente Hiperbólica (Tanh)

Definición matemática:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1)$$

¿Qué hace? Transforma las entradas a un rango acotado entre $[-1, 1]$. A diferencia de la función sigmoide (cuyo rango es $[0, 1]$), la función Tanh está **centrada en cero**, lo que facilita el proceso de optimización mediante el descenso de gradiente, haciendo que las actualizaciones de los pesos sean más estables y rápidas durante el entrenamiento en capas ocultas.

1.2. 2. Leaky ReLU (Leaky Rectified Linear Unit)

Definición matemática:

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases} \quad (\text{donde } \alpha \text{ suele ser } 0,01) \quad (2)$$

¿Qué hace? Resuelve directamente el problema de las *"neuronas muertas"* que sufre la función ReLU tradicional. Al asignar una pequeña pendiente constante (α) a las entradas negativas, permite que la neurona siga propagando un gradiente no nulo durante el *backpropagation*, manteniendo activas las neuronas incluso cuando reciben valores de entrada negativos.

1.3. 3. GELU (Gaussian Error Linear Unit)

Definición matemática:

$$f(x) = x \cdot \Phi(x) = x \cdot P(X \leq x), \quad X \sim \mathcal{N}(0, 1) \quad (3)$$

¿Qué hace? Pondera las entradas según una distribución probabilística gaussiana. A diferencia de ReLU o LeakyReLU, GELU es una función completamente suave (derivable en todo su dominio) que combina propiedades de activación aleatoria (dropouts) y no-linealidad estocástica. Es ampliamente utilizada en arquitecturas modernas de procesamiento de lenguaje natural y visión, como BERT, GPT y ViT (Vision Transformers).

2. Implementación en Python

El siguiente script en Python utiliza la biblioteca **numpy** para calcular las funciones de activación e implementa un modelo sencillo con la API de Keras utilizando la función **LeakyReLU**.

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras import layers, models
4
5 # 1. Definición matemática de las funciones de activación
6 def tanh(x):
7     return np.tanh(x)
8
9 def leaky_relu(x, alpha=0.01):
10     return np.where(x > 0, x, x * alpha)
11
12 def gelu(x):
13     return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715 * np.power(
14         x, 3))))
15
16 # 2. Ejemplo de prueba con datos numéricos
17 x_input = np.array([-3.0, -1.0, 0.0, 1.0, 3.0])
18
19 print("Entrada:", x_input)
20 print("Tanh:", np.round(tanh(x_input), 4))
21 print("LeakyReLU:", np.round(leaky_relu(x_input), 4))
22 print("GELU:", np.round(gelu(x_input), 4))
23
24 # 3. Integración en un modelo de Keras / TensorFlow
25 model = models.Sequential([
26     layers.Input(shape=(4,)),
27     layers.Dense(16),
28     layers.LeakyReLU(alpha=0.01), # Capa con activación LeakyReLU
29     layers.Dense(8, activation='gelu'), # Capa con activación GELU
30     layers.Dense(1, activation='tanh') # Capa de salida con activación Tanh
31 ])
32 model.summary()
```

Listing 1: Demostración de funciones de activación en Python

3. Conclusión

La elección de la función de activación depende del problema específico y la posición de la capa dentro de la red. Mientras que **Tanh** ayuda en redes donde se requiere una salida centrada en cero, **LeakyReLU** mejora la estabilidad frente a ReLU, y **GELU** proporciona un desempeño superior en modelos avanzados de aprendizaje profundo.